

Creación de un Sistema Agéntico

En esta sesión vamos a construir, un planificador conversacional de itinerarios de viaje. El usuario chatea; el agente planifica, modifica y renderiza un panel de itinerario en vivo. El usuario también puede eliminar actividades con un botón y el agente puede añadir/quitar actividades.

Cuando veas un comentario # **TODO:** o **// TODO:**, deberás escribir tú mismo la línea de código faltante. Lee atentamente el contexto y las explicaciones antes y después del bloque para deducir lo que hace falta. ¡No te saltes los checkpoints! Te dirán si vas por buen camino.

Tabla de Contenido

Arquitectura	2
PARTE A — Backend	2
A1. Configuración del Proyecto	2
A2. Un Grafo Vacío	3
A3. El Servidor	4
A4. Esquema del Estado	5
A5. El Nodo classify	7
A6. El Nodo plan	9
A7. Enrutamiento + Nodo respond	10
A8. El Nodo modify	12
A9. El Nodo ask	13
PARTE B — Frontend	14
B1. Configuración del Proyecto	14
B2. Conectar CopilotKit	15
B3. La Ruta del Runtime	15
B4. El Panel de Chat	16
B5. CSS de Layout	17
B6. Estado Compartido con useAgent	19
B7. Renderizar el Itinerario	20
B8. Generative UI: la tarjeta resumen del viaje	23
B9. Registrarlo como herramienta con useComponent	24

Arquitectura

Para la creación de este agente, vamos a construir un grafo con LangGraph. Este grafo funcionará esencialmente como un clasificador de intención con ramificaciones. Utilizaremos el protocolo AG-UI como un puente esencial para transmitir el estado del grafo al frontend.

En el frontend, se implementará CopilotKit. Este framework proporcionará la interfaz de chat, gestión del estado compartido del agente y herramientas propias del frontend. CopilotKit facilita que los cambios de estado del grafo se reflejen automáticamente en React a través del hook useCoAgent, eliminando la necesidad de realizar llamadas a APIs de forma manual.

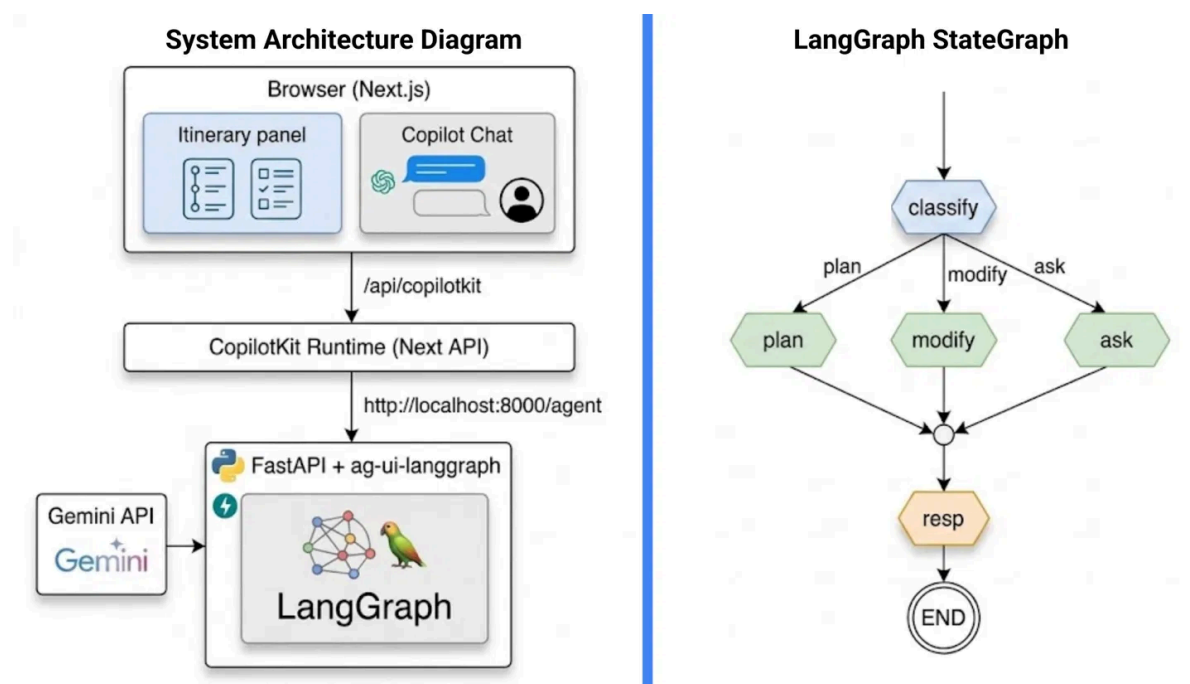


Fig. 1. Arquitectura.

PARTE A — Backend

A1. Configuración del Proyecto

Inicializa el proyecto Python con UV. Si requieres instalar UV, sigue las [instrucciones oficiales](#):

```
Shell
uv init itinerary-agent
cd itinerary-agent
uv add langgraph "langchain-google-genai" "ag-ui-langgraph" "copilotkit" fastapi
"uvicorn[standard]" python-dotenv pydantic
```

Crea un archivo **.env** en la raíz del proyecto. Si requieres generar un API KEY, puedes entrar a [Google AI Studio](#). La capa gratuita de **gemini-3.1-flash-lite-preview** es suficiente para toda la sesión.

```
Shell
GOOGLE_API_KEY=tu-clave-gratuita-de-gemini
```

Elimina el **main.py** de plantilla que **uv init** creó.

Checkpoint: Te debe imprimir "ok".

Shell

```
uv run python -c "import langgraph, fastapi; print('ok')"
```

A2. Un Grafo Vacío

Empezamos con el grafo más pequeño posible: un nodo que aún no hace nada útil. El objetivo es ver el grafo compilarse e invocarse antes de añadir inteligencia.

Crea el archivo **agent.py**:

Python

```
from typing import Annotated, Sequence
from copilotkit import CopilotKitState
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.messages import BaseMessage, HumanMessage, AIMessage
from dotenv import load_dotenv

load_dotenv()

class ItineraryState(CopilotKitState):
    messages: Annotated[Sequence[BaseMessage], add_messages]

def respond_node(state: ItineraryState) -> dict:
    # TODO: Devuelve un diccionario con la clave "messages" que contenga
    # una lista con un AIMessage cuyo contenido sea "Hello from the graph!"

builder = StateGraph(ItineraryState)
builder.add_node("respond", respond_node)
# TODO: Añade una arista (edge) desde START hasta el nodo "respond"
# TODO: Añade una arista (edge) desde el nodo "respond" hasta END

graph = builder.compile(checkpointer=InMemorySaver())

if __name__ == "__main__":
    cfg = {"configurable": {"thread_id": "smoke-1"}}
    r = graph.invoke(
        {"messages": [HumanMessage(content="hi")]},
        config=cfg,
    )
    print(r["messages"][-1].content)
```

Recuerda estos puntos clave:

- **StateGraph** y **CopilotKitState** definen el estado. Piensa en el estado como una bolsa de datos que pasa de un nodo a otro. Heredamos de **CopilotKitState** (en lugar de **TypedDict** puro) porque eso nos agrega de forma transparente los campos que CopilotKit usa para sincronizarse con el frontend.
- El reducer `add_messages` no reemplaza la conversación anterior, sino que añade los nuevos mensajes. Es el reducer más importante de LangGraph para mantener el hilo.
- **START** y **END** son como el inicio y el final de nuestro programa.
- El checkpointer **InMemorySaver** nos da "memoria" por conversación, usando el `thread_id` para recordar el historial.
- Un nodo es simplemente una función que toma el estado y devuelve un diccionario con los cambios que quieres aplicarle a ese estado.

Checkpoint: Te debe imprimir "Hello from the graph!"

Shell

```
uv run python agent.py
```

A3. El Servidor

Ahora exponemos el grafo por HTTP usando el adaptador AG-UI. Crea **server.py**:

Python

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from ag_ui_langgraph import LangGraphAgent, add_langgraph_fastapi_endpoint
from agent import graph

app = FastAPI()

# Permite que Next.js (puerto 3000) llame al backend en dev.
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

agent = LangGraphAgent(
    name="itinerary_agent",
    description="Plans and modifies travel itineraries through conversation",
    graph=graph,
)

# TODO: Llama a add_langgraph_fastapi_endpoint pasando app, agent y path="/agent"
```

LangGraphAgent envuelve nuestro grafo compilado y habla el protocolo AG-UI (un stream de eventos que el frontend entiende). Un agente tiene un nombre único; el frontend lo referenciará por este nombre exacto (**itinerary_agent**). La función **add_langgraph_fastapi_endpoint** monta un endpoint POST con streaming en el path **/agent**.

El middleware CORS evita que el navegador bloquee las llamadas durante el desarrollo. En producción, el CopilotKit Runtime se encarga, pero en el entorno de desarrollo este header te ahorrará tiempo si haces tests directos al backend.

Checkpoint: Inicia el servidor y navega a <http://127.0.0.1:8000/docs>. El servidor no debe tener errores.

Shell

```
uv run uvicorn server:app --reload --port 8000
```

A4. Esquema del Estado

Ahora diseñamos los datos. El agente necesita datos estructurados del itinerario, no solo mensajes de chat. Añade los imports y los modelos Pydantic, reemplazando la parte superior de **agent.py**:

Python

```
import json
from typing import Annotated, List, Literal, Optional, Sequence

from copilotkit import CopilotKitState
from dotenv import load_dotenv
from langchain_core.messages import (
    AIMessage,
    BaseMessage,
    HumanMessage,
    SystemMessage
)
from langchain_google_genai import ChatGoogleGenerativeAI
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import END, START, StateGraph
from langgraph.graph.message import add_messages
from pydantic import BaseModel, Field

load_dotenv()

MODEL = "gemini-3.1-flash-lite"
llm = ChatGoogleGenerativeAI(model=MODEL)
```

Añade los modelos del dominio:

Python

```
class Activity(BaseModel):
    time: str = Field(description="Formato HH:MM 24h")
    title: str
    description: str
    location: str
```

```

class DayPlan(BaseModel):
    day: int
    summary: str
    activities: List[Activity]

class ItineraryPlan(BaseModel):
    # TODO: Define un campo "days" que sea una lista (List) de DayPlan

class Intent(BaseModel):
    intent: Literal["new_plan", "modify", "missing_info", "chat"] = Field(
        description=(
            "new_plan: el usuario quiere un itinerario nuevo y tanto la ciudad "
            "como los días están claros. "
            "modify: el usuario quiere cambiar el itinerario existente. "
            "missing_info: el usuario quiere un plan pero falta la ciudad o los días. "
            "chat: pregunta general o conversación."
        )
    )
    city: Optional[str] = Field(None, description="Ciudad de destino si se menciona")
    num_days: Optional[int] = Field(None, description="Número de días si se menciona")
    missing: Optional[str] = Field(
        None,
        description="Qué falta, p. ej. 'la ciudad de destino'. Solo para missing_info.",
    )
    modification: Optional[str] = Field(
        None, description="Modificación exacta solicitada. Solo para modify."
    )

```

Expande el estado:

Python

```

class ItineraryState(CopilotKitState):
    messages: Annotated[Sequence[BaseMessage], add_messages]
    # TODO: Añade los campos "city" (Optional[str]) y "num_days" (Optional[int])
    # TODO: Añade el campo "itinerary" (List[DayPlan])
    # TODO: Añade el campo "intent"
    #         (Optional[Literal["new_plan", "modify", "missing_info", "chat"]])
    # TODO: Añade los campos "missing" (Optional[str]) y "modification" (Optional[str])

```

Imagina que estás hablándole a alguien sobre cómo funciona una app inteligente de planificación. La **salida estructurada** es pedirle al modelo que, en lugar de contarte una historia, llene un formulario específico. La magia es que te entrega la información ya organizada en un formato perfecto para que otro programa la use (como un JSON), sin que tengas que limpiar el texto después.

La **intención** funciona como el cerebro de la app que te escucha y clasifica. Cuando le dices algo, lo primero que hace es meter lo que dijiste en una de las cuatro categorías definidas. Pero no solo clasifica: de una vez extrae los datos clave (ciudad, número de días, modificación, etc.).

Lo que escribimos en la **descripción** del campo (**Field(description=...)**) son instrucciones precisas que le damos al modelo. No es solo para que nosotros sepamos qué va ahí: funciona como una guía interna detallada que le dice al modelo exactamente qué tipo de información generar en ese punto.

Finalmente, el **estado de la aplicación** es la memoria completa. Guarda dos cosas fundamentales: el historial de la conversación y, de forma organizada, los datos clave (itinerario, ciudad, etc.). Lo que ves en pantalla es un reflejo de lo que está guardado en ese estado. Por heredar de **CopilotKitState**, este estado también se sincronizará automáticamente con el frontend.

Ahora, añade las variantes del LLM con salida estructurada en **agent.py**:

Python

```
# TODO: Crea una variable "classifier" llamando a llm.with_structured_output(Intent)
# TODO: Crea una variable "planner" llamando a llm.with_structured_output(ItineraryPlan)
```

.with_structured_output(Model) hace que el LLM devuelva un objeto Pydantic validado en lugar de texto libre. Esta es la columna vertebral de todo el agente.

Checkpoint: Aún no hay cambio de comportamiento, pero sí debemos confirmar que el esquema compila.

Shell

```
uv run python -c "import agent; print('imports ok')"
```

A5. El Nodo classify

Este es el primer nodo de "IA real". Mira la conversación y decide qué quiere el usuario.

Python

```
CLASSIFY_SYSTEM_PROMPT = (
    "Eres un clasificador de asistente de viajes.\n"
    "Ciudad actual: {city}\n"
    "Días actuales: {num_days}\n"
    "Existe itinerario: {has_itinerary}\n\n"
    "Conversación hasta ahora:\n{history}\n\n"
    "Clasifica la última intención del usuario."
)

CLASSIFY_HUMAN_PROMPT = 'Clasifica esta solicitud: "{request}"'

def classify_node(state: ItineraryState) -> dict:
    humans = [m for m in state["messages"] if isinstance(m, HumanMessage)]
    if not humans:
        return {"intent": "chat", "missing": None, "modification": None}

    history_summary = "\n".join(
        f"{'User' if isinstance(m, HumanMessage) else 'Assistant'}: {m.content}"
        for m in state["messages"]
        if hasattr(m, "content") and m.content
    )

    result: Intent = classifier.invoke([
        SystemMessage(content=CLASSIFY_SYSTEM_PROMPT.format(
            city=state.get("city") or "not set",
```

```

        num_days=state.get("num_days") or "not set",
        has_itinerary=bool(state.get("itinerary")),
        history=history_summary,
    )),
    HumanMessage(content=CLASSIFY_HUMAN_PROMPT.format(
        request=humans[-1].content)),
])

return {
    # TODO: Devuelve "city" con valor result.city o el state.get("city") existente
    # TODO: Devuelve "num_days" con valor result.num_days o
    #       el state.get("num_days") existente
    "intent": result.intent,
    "missing": result.missing,
    "modification": result.modification,
}

```

Le damos al modelo toda la información que necesita: la **ciudad**, cuántos **días**, si ya hay **plan** y todo lo conversado. En estos sistemas, la conversación y el contexto son fundamentales.

Además nos aseguramos de que el agente no olvide los detalles importantes mencionados antes. Si el usuario dijo la ciudad hace un rato y luego solo comenta "que sean 3 días", el agente entiende que esos 3 días son para la ciudad ya conocida.

Y para hacerlo más rápido y barato, en una sola llamada el modelo clasifica qué pide el usuario y también extrae los datos importantes. Ahora, reconecta el grafo temporalmente para probar solo este nodo:

Python

```

builder = StateGraph(ItineraryState)
builder.add_node("classify", classify_node)
builder.add_edge(START, "classify")
builder.add_edge("classify", END)
graph = builder.compile(checkpointer=InMemorySaver())

if __name__ == "__main__":
    cfg = {"configurable": {"thread_id": "smoke-1"}}
    r = graph.invoke(
        {"messages": [HumanMessage(content="Planea un viaje de dos dias a Lisboa")]},
        config=cfg,
    )
    print("Intent:", r["intent"], "| City:", r["city"], "| Days:", r["num_days"])

```

Checkpoint: Debe salirte "Intent: new_plan | City: Lisboa | Days: 2"

Shell

```
uv run python agent.py
```


A6. El Nodo plan

Ahora generamos un itinerario real usando salida estructurada.

Python

```
PLAN_SYSTEM_PROMPT = (
    "Crea un itinerario de viaje de {num_days} días para {city}. "
    "Incluye 3-4 actividades reales y conocidas por día con horarios realistas."
)
PLAN_HUMAN_PROMPT = "Genera el itinerario ahora."

def plan_node(state: ItineraryState) -> dict:
    # TODO: Invoca al planner pasando una lista con un SystemMessage formateado
    # con PLAN_SYSTEM_PROMPT (usando state["num_days"] y state["city"]) y un
    # HumanMessage con PLAN_HUMAN_PROMPT. Guarda el resultado en una variable
    # llamada "result" tipada como ItineraryPlan.

    # TODO: Devuelve un diccionario con la clave "itinerary" cuyo valor sea
    # [d.model_dump() for d in result.days]
    # (lo serializamos a dicts para que CopilotKit lo pueda enviar al frontend)
```

El protocolo AG-UI envía el estado al frontend como JSON, así que devolvemos dicts (model_dump()) en lugar de objetos Pydantic. Esto evita errores al cruzar la frontera entre servidor y cliente (y de vuelta).

Dado que **planner** es un **llm.with_structured_output(ItineraryPlan)**, se fuerza al modelo a devolver días y actividades coincidentes con nuestro esquema. El nodo devuelve solo la porción itinerario del estado y LangGraph la fusiona con el resto.

Ahora, prueba la cadena hasta ahora: **START** → **classify** → **plan** → **END**:

Python

```
builder = StateGraph(ItineraryState)
builder.add_node("classify", classify_node)
builder.add_node("plan", plan_node)
builder.add_edge(START, "classify")
builder.add_edge("classify", "plan")
builder.add_edge("plan", END)
graph = builder.compile(checkpointer=InMemorySaver())

if __name__ == "__main__":
    cfg = {"configurable": {"thread_id": "smoke-1"}}
    r = graph.invoke(
        {"messages": [HumanMessage(content="Planea un viaje de dos días a Lisboa")]},
        config=cfg,
    )
    print("Days generated:", len(r["itinerary"]))
    print(r["itinerary"][0])
```

Checkpoint: Te debe devolver un itinerario de dos días en Lisboa.

Shell

```
uv run python agent.py
```

A7. Enrutamiento + Nodo respond

Un clasificador es inútil sin ramificación. Añade el enrutador condicional y el nodo conversacional **respond**.

Python

```
RESPOND_TASK_NEW_PLAN = (
    "Acabas de generar un itinerario de {num_days} días para {city}. "
    "Dile al usuario que está listo en una frase amistosa. "
    "Menciona que puede eliminar actividades con los botones ✕ o pedir cambios."
)
RESPOND_TASK_MODIFY = (
    'Acabas de aplicar este cambio: "{modification}". '
    "Confirma lo que cambiaste en una frase amistosa."
)
RESPOND_TASK_DEFAULT = "Responde la pregunta del usuario de forma útil y concisa."

RESPOND_SYSTEM_PROMPT = (
    "Eres un asistente amable de planificación de viajes.\n"
    "Ciudad actual: {city}\n"
    "Días actuales: {num_days}\n"
    "Existe itinerario: {has_itinerary}\n\n"
    "Tarea: {task}"
)
RESPOND_NO_HUMAN_REPLY = "¿Cómo puedo ayudarte a planear tu viaje?"

def respond_node(state: ItineraryState) -> dict:
    intent = state.get("intent")
    if intent == "new_plan":
        task = RESPOND_TASK_NEW_PLAN.format(
            num_days=state["num_days"], city=state["city"])
    elif intent == "modify":
        task = RESPOND_TASK_MODIFY.format(
            modification=state.get("modification"))
    else:
        task = RESPOND_TASK_DEFAULT

    humans = [m for m in state["messages"] if isinstance(m, HumanMessage)]
    if not humans:
        return {"messages": [AIMessage(content=RESPOND_NO_HUMAN_REPLY)]}

    reply = llm.invoke([
        SystemMessage(content=RESPOND_SYSTEM_PROMPT.format(
            city=state.get("city") or "not set",
            num_days=state.get("num_days") or "not set",
            has_itinerary=bool(state.get("itinerary")),
            task=task,
        )),
```

```

        HumanMessage(content=humans[-1].content),
    ])
    return {"messages": [AIMessage(content=reply.content)]}

```

La función de enrutamiento:

Python

```

def route_after_classify(state: ItineraryState) -> str:
    intent = state.get("intent")
    # TODO: Si intent es "new_plan", devuelve "plan"
    # TODO: Si intent es "modify", devuelve "modify"
    # TODO: Si intent es "missing_info", devuelve "ask"
    # TODO: En cualquier otro caso, devuelve "respond"

```

Reconecta con aristas condicionales (nota: los nodos **modify** y **ask** aún no existen: añadiremos funciones vacías por el momento):

Python

```

def modify_node(state): return {}
def ask_node(state): return {}

builder = StateGraph(ItineraryState)
builder.add_node("classify", classify_node)
builder.add_node("plan", plan_node)
builder.add_node("modify", modify_node)
builder.add_node("ask", ask_node)
builder.add_node("respond", respond_node)

builder.add_edge(START, "classify")
# TODO: Añade aristas condicionales (add_conditional_edges) desde "classify"
# usando route_after_classify como función de enrutamiento, con la lista
# ["plan", "modify", "ask", "respond"] como destinos posibles.
builder.add_edge("plan", "respond")
builder.add_edge("modify", "respond")
builder.add_edge("ask", END)
builder.add_edge("respond", END)

graph = builder.compile(checkpointer=InMemorySaver())

```

Cuando necesitamos que un agente decida su propio camino en un flujo, usamos la función **add_conditional_edges**. Básicamente, le decimos: "Ejecuta esta función (fn), y lo que devuelva será el nombre del siguiente paso a seguir, eligiendo entre estas posibles opciones".

Es importante resaltar qué los comandos como **ask** terminan el proceso y van a **END**, mientras que comandos como **plan** o **modify** van a **respond**. La razón es simple: cuando usamos **ask**, el agente ya ha generado un mensaje completo para el usuario por sí mismo. En cambio, si usamos **plan** o **modify**, aún necesitamos un paso adicional, que es **respond**, para que este formule una confirmación o una respuesta final para el usuario.

Además, a diferencia de los otros pasos que suelen tener estructuras de datos más rígidas, **respond** utiliza una invocación simple al LLM para generar texto libre.

Checkpoint: Valida que el grafo sigue funcionando y generando un itinerario

Python

```
uv run python agent.py
```

A8. El Nodo modify

Reemplaza la función **modify_node** con la real.

Python

```
MODIFY_SYSTEM_PROMPT = (
    "Aquí está el itinerario actual de {num_days} días para {city} en JSON:\n"
    "{itinerary}\n\n"
    "Aplica la modificación solicitada y devuelve el itinerario completo "
    "actualizado, conservando todos los días y actividades sin cambios."
)

MODIFY_HUMAN_PROMPT = "Modificación: {modification}"

def modify_node(state: ItineraryState) -> dict:
    # El estado puede llegar como dicts (desde el frontend) o como objetos Pydantic
    # (desde el propio grafo). Normalizamos siempre a DayPlan para validar.
    raw = state.get("itinerary") or []
    current = [DayPlan.model_validate(d) for d in raw]

    itinerary_json = (
        json.dumps([d.model_dump() for d in current], indent=2)
        if current else "none"
    )

    result: ItineraryPlan = planner.invoke([
        SystemMessage(content=MODIFY_SYSTEM_PROMPT.format(
            num_days=state.get("num_days"),
            city=state.get("city"),
            itinerary=itinerary_json,
        )),
        HumanMessage(content=MODIFY_HUMAN_PROMPT.format(
            modification=state.get("modification")
        )),
    ])

    # TODO: Devuelve un diccionario con la clave "itinerary" cuyo valor sea
    # la lista de actividades serializadas: [d.model_dump() for d in result.days]
```

Serializamos el itinerario actual de vuelta a JSON y se lo damos al modelo con el cambio solicitado. El modelo devuelve el itinerario completo actualizado (más simple que pedirle un diff). Adicionalmente, usamos **Day**

Plan.model validate(...) como una defensa, por si el estado entrante son diccionarios planos (desde el frontend) en lugar de objetos Pydantic.

Checkpoint: Prueba el multi-turno en el mismo hilo:

Python

```
if __name__ == "__main__":
    cfg = {"configurable": {"thread_id": "smoke-1"}}

    r = graph.invoke(
        {"messages": [HumanMessage(content="Viaje de 2 dias a Lisboa")]},
        config=cfg,
    )
    print("Intent:", r["intent"])
    print("Day 1 activities:", len(r["itinerary"][0]["activities"]))

    r = graph.invoke(
        {"messages": [HumanMessage(content="Quita la primera actividad del dia 1")]},
        config=cfg,
    )
    print("Intent:", r["intent"])
    print("Day 1 activities now:", len(r["itinerary"][0]["activities"]))
```

Valida que el grafo genera un itinerario y luego elimina una actividad.

Shell

```
uv run python agent.py
```

A9. El Nodo ask

Reemplaza la función de **ask_node**.

Python

```
ASK_REPLY = (
    "¡Me encantaría ayudarte a planear tu viaje! ¿Podrías decirme {missing}?"
)
ASK_DEFAULT_MISSING = "algunos detalles más"

def ask_node(state: ItineraryState) -> dict:
    # TODO: Devuelve un diccionario con la clave "messages" que contenga una lista
    # con un AIMessage. El contenido del AIMessage debe ser ASK_REPLY formateado
    # con missing=state.get("missing") o, si no existe, ASK_DEFAULT_MISSING.
```

Es importante resaltar que **no todos los nodos necesitan un LLM**. Cuando el clasificador dice **missing_info** y rellena el campo **missing**, pedimos la información de forma determinista. Barato, rápido, predecible. Un buen diseño de agente: usar un LLM solo donde necesites razonamiento.

Checkpoint: El backend completo se comporta correctamente en todas intenciones:

Python

```
if __name__ == "__main__":
    for thread, msg in [
        ("t1", "Planea un viaje de 3 dias a Roma"),          # new_plan
        ("t2", "Quiero un viaje"),                          # missing_info
        ("t3", "¿Cuál es la mejor época para ir a Japón?"), # chat
    ]:
        cfg = {"configurable": {"thread_id": thread}}
        r = graph.invoke({"messages": [HumanMessage(content=msg)]}, config=cfg)
        print(thread, "→", r["intent"], "|", r["messages"][-1].content)
```

Valida que el grafo es capaz de detectar la intención correcta en cada caso.

Shell

```
uv run python agent.py
```

El backend está listo. Mantenlo corriendo en el puerto 8000.

Shell

```
uv run uvicorn server:app --reload --port 8000
```

PARTE B — Frontend

B1. Configuración del Proyecto

En una nueva terminal, desde la carpeta padre (al lado de **itinerary-agent**):

Shell

```
npx create-next-app@latest frontend --typescript --app --no-tailwind --eslint --no-src-dir
--import-alias "@/*"
cd frontend
npm install @copilotkit/react-core @copilotkit/runtime
```

Checkpoint: Prueba la App Vacía. Siempre confirma que el esqueleto funciona antes de añadir nada.

Shell

```
npm run dev
```

Abre <http://localhost:3000> deberías ver la página por defecto de Next.js.

B2. Conectar CopilotKit

CopilotKit es un proveedor de contexto de React. Todo lo que hable con el agente debe vivir dentro de él, por lo que va en el layout raíz. Reemplaza **app/layout.tsx**:

TypeScript

```
import { CopilotKit } from "@copilotkit/react-core/v2";
import "@copilotkit/react-core/v2/styles.css";

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html>
      <body>
        { /* TODO: Envuelve {children} con el componente <CopilotKit>.
           Debes pasarle dos props:
           - runtimeUrl="/api/copilotkit"
           - agent="itinerary_agent"
           (debe coincidir EXACTAMENTE con el name= de server.py) */ }
        {children}
      </body>
    </html>
  );
}
```

El **runtimeUrl="/api/copilotkit"** indica que el frontend nunca se comunica directamente con el backend de Python, sino que utiliza una ruta API de Next.js (que se construirá más adelante) como proxy hacia LangGraph. El parámetro **agent="itinerary_agent"** debe coincidir exactamente con el valor del atributo **name** en **server.py**; este es un punto crucial para evitar el error clásico de discrepancia. Finalmente, el CSS de CopilotKit se importa una sola vez en este lugar.

B3. La Ruta del Runtime

Este es el puente Next.js entre LangGraph. Crea **app/api/copilotkit/route.ts**:

TypeScript

```
import {
  CopilotRuntime,
  ExperimentalEmptyAdapter,
  copilotRuntimeNextJSAppRouterEndpoint,
} from "@copilotkit/runtime";
import { LangGraphHttpAgent } from "@copilotkit/runtime/langgraph";
import { NextRequest } from "next/server";

const serviceAdapter = new ExperimentalEmptyAdapter();

const runtime = new CopilotRuntime({
  agents: {
    // TODO: Define la clave "itinerary_agent" como un nuevo LangGraphHttpAgent.
    // Su constructor recibe un objeto con la clave url: "http://localhost:8000/agent"
  },
});
```

```
});

export const POST = async (req: NextRequest) => {
  const { handleRequest } = copilotRuntimeNextJSAppRouterEndpoint({
    runtime,
    serviceAdapter,
    endpoint: "/api/copilotkit",
  });
  return handleRequest(req);
};

export const GET = POST; // GET se usa para /info; reutilizamos el mismo handler.
```

La configuración utiliza **LangGraphHttpAgent({ url: ".../agent" })** para dirigir al endpoint de FastAPI, y el **ExperimentalEmptyAdapter** actúa como un simple marcador de posición requerido, ya que no se utiliza un modelo de lenguaje grande (LLM) separado en la capa de ejecución, pues toda la inteligencia reside en el grafo de Python. Exportamos **GET = POST** porque el CopilotKit Runtime expone un endpoint **/info (GET)** que sincroniza la lista de agentes disponibles con el frontend.

Checkpoint: Válida que <http://localhost:3000/api/copilotkit> retorna cualquier código HTTP no-404. Esto confirma que la ruta existe y está conectada

```
Shell
npm run dev
```

B4. El Panel de Chat

Ahora la UI real. Empieza mínimo: solo el chat, sin panel de itinerario todavía. Crea **app/page.tsx**:

```
TypeScript
"use client";

import { CopilotChat } from "@copilotkit/react-core/v2";

export default function Page() {
  return (
    <div style={{ height: "100vh", width: 400, margin: "0 auto" }}>
      <CopilotChat
        agentId="itinerary_agent"
        labels={{ chatInputPlaceholder: "¿A dónde quieres viajar?" }}
        welcomeScreen={({ input }) => (
          <div style={{ padding: 24, textAlign: "center" }}>
            <p>¿A dónde te gustaría ir?</p>
            {input}
          </div>
        )}
      />
    </div>
```



```
);  
}
```

La directiva **"use client"** es necesaria porque los hooks de CopilotKit requieren el navegador, El parámetro **agentId="itinerary_agent"** selecciona explícitamente el agente. Aunque el provider ya lo configura como **agent=...**, ser explícito ayuda cuando tengas varios agentes. Finalmente, **welcomeScreen** es una render prop. El parámetro **input** es la caja de texto que provee CopilotKit: la insertas donde quieras dentro del **welcome**.

Checkpoint: Valida que <http://localhost:3000/> muestra el chat de CopilotKit y que puedes mandar un mensaje y recibir respuesta del agente.

```
Shell  
npm run dev
```

B5. CSS de Layout

Crea **app/page.module.css** con las piezas estructurales (split raíz + chat + itinerario):

```
CSS  
.root {  
  display: flex;  
  height: 100vh;  
  font-family: system-ui, sans-serif;  
  color: #111;  
}  
  
.panel {  
  flex: 1;  
  overflow-y: auto;  
  padding: 24px 32px;  
  border-right: 1px solid #e5e7eb;  
}  
  
.heading {  
  font-size: 1.5rem;  
  font-weight: 700;  
  margin-bottom: 24px;  
}  
  
.chat {  
  width: 400px;  
  height: 100vh;  
  display: flex;  
  flex-direction: column;  
  overflow: hidden;  
}
```

```

.welcome {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  height: 100%;
  gap: 8px;
  padding: 24px;
  text-align: center;
}

.welcomeTitle {
  font-size: 1.1rem;
  font-weight: 600;
  color: #111;
  margin: 0;
}

```

Actualiza **page.tsx** para usar el layout (placeholder de panel izquierdo + chat a la derecha):

```

TypeScript
"use client";

import { CopilotChat } from "@copilotkit/react-core/v2";
import styles from "../page.module.css";

export default function Page() {
  return (
    <div className={styles.root}>
      <div className={styles.panel}>
        <h1 className={styles.heading}>Trip Planner</h1>
      </div>
      <div className={styles.chat}>
        <CopilotChat
          agentId="itinerary_agent"
          labels={{ chatInputPlaceholder: "¿A dónde quieres viajar?" }}
          welcomeScreen={({ input }) => (
            <div className={styles.welcome}>
              <p className={styles.welcomeTitle}>¿A dónde te gustaría ir?</p>
              {input}
            </div>
          )}
        />
      </div>
    </div>
  );
}

```

Checkpoint: Prueba el Chat. Esta es la primera prueba real full-stack. Ambos servidores deben estar corriendo:

- Terminal 1 en itinerary-agent: **uv run uvicorn server:app --reload --port 8000**
- Terminal 2 en frontend: **npm run dev**

Abre <http://localhost:3000> y escribe:

None

Planea un viaje de 3 días a Barcelona

Deberías recibir una respuesta amistosa del asistente en el chat. El itinerario aún no se renderiza en el panel, pero el agente sí lo generó del lado del servidor.

B6. Estado Compartido con useAgent

Aquí está la magia de CopilotKit + LangGraph: el estado del grafo se refleja en React sin fetching manual. Actualiza **page.tsx** añadiendo los tipos y el hook **useAgent**:

Python

```
"use client";

import { CopilotChat, useAgent } from "@copilotkit/react-core/v2";
import styles from "./page.module.css";

type Activity = {
  time: string;
  title: string;
  description: string;
  location: string;
};

type DayPlan = {
  day: number;
  summary: string;
  activities: Activity[];
};

export default function Page() {
  // TODO: Llama al hook useAgent pasándole un objeto con agentId: "itinerary_agent".
  // Desestructura "agent" del resultado.

  const itinerary = (agent.state.itinerary as DayPlan[] | undefined) ?? [];
  const hasItinerary = itinerary.length > 0;
  const city = (agent.state.city as string | undefined) ?? "";
  const numDays = (agent.state.num_days as number | undefined) ?? 0;

  return (
    <div className={styles.root}>
      <div className={styles.panel}>
        <h1 className={styles.heading}>
          {hasItinerary ? `${numDays}-Day Itinerary: ${city}` : "Trip Planner"}
        </h1>

        {/* Debug útil mientras desarrollas - quítalo cuando todo funcione */}
        <pre style={{ fontSize: 11, color: "#999" }}>
          {JSON.stringify(agent.state, null, 2)}
        </pre>
      </div>
    </div>
  );
}
```

```

    </pre>
  </div>
  <div className={styles.chat}>
    <CopilotChat
      agentId="itinerary_agent"
      labels={{ chatInputPlaceholder: "¿A dónde quieres viajar?" }}
      welcomeScreen={({ input }) => (
        <div className={styles.welcome}>
          <p className={styles.welcomeTitle}>¿A dónde te gustaría ir?</p>
          {input}
        </div>
      )}
    />
  </div>
</div>
);
}

```

Un **useAgent** devuelve un objeto agent con **agent.state**, **agent.messages**, **agent.isRunning** y métodos como **agent.setState({...})** y **agent.runAgent()**. Es la forma idiomática de conectar React con el agente.

Checkpoint: Pide un plan en el chat, y mira el panel izquierdo llenándose con el estado actualizando en tiempo real.

B7. Renderizar el Itinerario

Entre la definición de **numDays** y el **return**, crear los handlers para actualizar el itinerario:

TypeScript

```

const updateItinerary = (mutate: (draft: DayPlan[]) => void) => {
  const prev = (agent.state.itinerary as DayPlan[] | undefined) ?? [];
  const next = structuredClone(prev);
  mutate(next);
  agent.setState({ itinerary: next });
};

const removeActivity = (dayIdx: number, actIdx: number) =>
  // TODO: Llama a updateItinerary pasándole una función que reciba "draft"
  // y use draft[dayIdx].activities.splice(actIdx, 1) para eliminar la actividad.

```

Reemplaza el bloque **<pre>** del panel en **page.tsx** con el render real:

TypeScript

```

{itinerary.map((d, dayIdx) => (
  <div key={d.day} className={styles.day}>
    <h2 className={styles.dayHeading}>
      Día {d.day} <span className={styles.daySummary}>- {d.summary}</span>
    </h2>
  </div>
)}

```

```

</h2>
{d.activities?.map((a, actIdx) => (
  <div key={actIdx} className={styles.activity}>
    <div className={styles.activityHeader}>
      <span className={styles.time}>{a.time}</span>
      <span className={styles.activityTitle}>{a.title}</span>
      <span className={styles.location}>📍 {a.location}</span>
      <button
        onClick={() => removeActivity(dayIdx, actIdx)}
        className={styles.removeBtn}
        title={`Quitar ${a.title}`}
      >
        ✕
      </button>
    </div>
    <p className={styles.description}>{a.description}</p>
  </div>
  )))
</div>
))}

```

Ahora, añade el CSS del itinerario al final de **page.module.css**.

```

CSS

.empty {
  margin-top: 80px;
  text-align: center;
  color: #555;
}

.hint {
  color: #888;
  font-size: 0.9em;
}

.day {
  margin-bottom: 32px;
}

.dayHeading {
  font-size: 1.1rem;
  font-weight: 600;
  margin-bottom: 12px;
  color: #1d4ed8;
}

.daySummary {
  font-weight: 400;
  color: #555;
  font-size: 0.95rem;
}

.activity {

```

```

background: #f9fafb;
border: 1px solid #e5e7eb;
border-radius: 8px;
padding: 10px 14px;
margin-bottom: 10px;
}

.activityHeader {
display: flex;
align-items: center;
gap: 10px;
flex-wrap: wrap;
}

.time {
font-weight: 600;
font-size: 0.85rem;
color: #374151;
background: #e0e7ff;
border-radius: 4px;
padding: 1px 6px;
}

.activityTitle {
flex: 1;
font-weight: 600;
}

.location {
color: #6b7280;
font-size: 0.85rem;
}

.removeBtn {
border: 1px solid #fca5a5;
border-radius: 4px;
background: none;
color: #ef4444;
cursor: pointer;
padding: 1px 7px;
font-size: 0.8rem;
}

.description {
margin: 6px 0 0;
color: #6b7280;
font-size: 0.875rem;
}

```

Checkpoint: Probar el Renderizado. Ambos servidores corriendo. Abre <http://localhost:3000>.

1. Pregunta: "Planea un viaje de 2 días a Bogotá". Deberías ver las tarjetas renderizadas en el panel izquierdo.
2. Haz clic en un botón X — esa actividad debe desaparecer inmediatamente.

3. Pide en el chat: "Elimina todos los museos". El itinerario completo se actualiza.

B8. Generative UI: la tarjeta resumen del viaje

Hasta aquí, el agente actualiza un panel "estático" controlado por nosotros. Ahora, vamos a darle al agente la capacidad de renderizar un componente React directamente en el chat cada vez que crea o modifica un itinerario. Ese componente será una **TripSummaryCard** que muestra de un vistazo: destino, número de días y total de actividades.

Este patrón se llama **Generative UI tool-based**: registramos un componente React con **useComponent** y CopilotKit lo expone al agente como una tool. Cuando el agente la llama, CopilotKit pinta el componente inline en el chat y le pasa los argumentos como **props** tipadas.

Crea **app/components/TripSummaryCard.tsx**:

```
TypeScript
"use client";

import { z } from "zod";

// Schema Zod = contrato con el agente. El agente verá este schema como
// los parámetros de la herramienta y CopilotKit los validará antes de
// pasártelos como props.
export const tripSummaryPropsSchema = z.object({
  city: z.string().describe("Ciudad de destino, p. ej. 'Lisboa'"),
  numDays: z.number().int().min(1).describe("Número de días del viaje"),
  totalActivities: z
    .number()
    .int()
    .min(0)
    .describe("Cantidad total de actividades planeadas en todo el viaje"),
  highlight: z
    .string()
    .optional()
    .describe(
      "Una frase corta y atractiva resumiendo el viaje, p. ej. " +
      "'Tres días entre museos y tapas en el corazón de Madrid'"
    ),
});

export type TripSummaryProps = z.infer<typeof tripSummaryPropsSchema>;

export function TripSummaryCard({
  city,
  numDays,
  totalActivities,
  highlight,
}: TripSummaryProps) {
  return (
    <div
      style={{
        border: "1px solid #c7d2fe",
        background: "linear-gradient(135deg,#eef2ff,#fef3c7)",
        borderRadius: 12,
```

```

padding: "16px 18px",
margin: "8px 0",
maxWidth: 320,
}}
>
<div style={{ fontSize: 12, opacity: 0.7, marginBottom: 4 }}>
  ✈ Itinerary ready
</div>
<div style={{ fontSize: 20, fontWeight: 700, marginBottom: 8 }}>
  {city}
</div>
<div style={{ display: "flex", gap: 16, fontSize: 14 }}>
  <span>
    <strong>{numDays}</strong> día{numDays === 1 ? "" : "s"}
  </span>
  <span>
    <strong>{totalActivities}</strong> actividad
    {totalActivities === 1 ? "" : "s"}
  </span>
</div>
{highlight && (
  <p style={{ marginTop: 10, fontSize: 13, color: "#374151" }}>
    {highlight}
  </p>
)}
</div>
);
}

```

El schema **Zod** es contractual. Lo que ahí escribas en **.describe(...)** se convierte literalmente en la documentación de la herramienta que ve el LLM. Sé claro y específico. También es importante remarcar que el componente es React puro. No sabe nada sobre CopilotKit, solo lee sus props. Eso lo hace fácil de testear de forma aislada. Finalmente, CopilotKit valida los argumentos del LLM contra el schema antes de llamar al componente, así que dentro del componente puedes asumir que los tipos son correctos.

B9. Registrarlo como herramienta con useComponent

Ahora dile a CopilotKit que registre tu componente como una herramienta visible para el agente. En **app/page.tsx**, añade el hook dentro del componente **Page** (después de **useAgent**):

TypeScript

```

export default function Page() {
  const { agent } = useAgent({ agentId: "itinerary_agent" });

  // TODO: Llama a useComponent con un objeto que tenga:
  //   - name: "show_trip_summary"
  //   - description: una frase pidiéndole al agente que llame esta tool
  //     "cada vez que cree o modifique un itinerario, para mostrar al usuario
  //     una tarjeta resumen del viaje"
  //   - parameters: tripSummaryPropsSchema
  //   - render: TripSummaryCard
  //

```



```
// Nota importante: el nombre debe ser un verbo, p. ej. show_trip_summary
// o render_trip_card. Eso ayuda al LLM a decidir cuándo invocarlo.

// ... resto del componente igual ...
}
```

No olvides añadir los **imports** correspondientes:

```
TypeScript
import { CopilotChat, useAgent, useComponent } from "@copilotkit/react-core/v2";
import {
  TripSummaryCard,
  tripSummaryPropsSchema,
} from "../components/TripSummaryCard";
```

Ahora actualiza el system prompt del nodo **respond** (en **agent.py**) para que sepa que esta herramienta existe. Reemplaza **RESPOND_SYSTEM_PROMPT**:

```
TypeScript
RESPOND_SYSTEM_PROMPT = (
  "Eres un asistente amable de planificación de viajes.\n"
  "Ciudad actual: {city}\n"
  "Días actuales: {num_days}\n"
  "Existe itinerario: {has_itinerary}\n\n"
  "Tarea: {task}\n\n"
  "Cuando acabes de crear o modificar un itinerario, llama a la herramienta "
  "`show_trip_summary` con `city`, `num_days`, el total de actividades "
  "(sumando todas las que hay en todos los días) y un `highlight` corto y "
  "evocador. Llámala UNA sola vez por respuesta."
)
```

Y, lo más importante, hay que darle al LLM la lista de tools del frontend. Las tools registradas con **useComponent** (o **useFrontendTool**) llegan al backend dentro del estado: **state["copilotkit"]["actions"]**. Modifica **respond_node** para bindearlas al modelo.

Pero hay un detalle sutil: cuando el agente llama a una tool del frontend (**show_trip_summary**), CopilotKit reporta el resultado al backend, lo que dispara una nueva ejecución del grafo con el **ToolMessage** añadido al historial. Si no manejamos eso, el grafo entra en bucle: clasifica de nuevo el último mensaje del usuario, vuelve a **modify** o **respond**, llama la tool otra vez, y así. Hay que cortar el ciclo en dos lugares:

Primero, asegúrate de importar **ToolMessage** al inicio de **agent.py**:

```
TypeScript
from langchain_core.messages import (
  AIMessage,
  BaseMessage,
  HumanMessage,
  SystemMessage,
  ToolMessage,
```

)

Después, modifica **classify_node** para ignorar las re-ejecuciones disparadas por un tool result. Añade este bloque justo después del chequeo de `if not humans`:

Python

```
def classify_node(state: ItineraryState) -> dict:
    humans = [m for m in state["messages"] if isinstance(m, HumanMessage)]
    if not humans:
        return {"intent": "chat", "missing": None, "modification": None}

    # Si el último mensaje es un ToolMessage, este run viene del resultado
    # de un frontend tool, no de un mensaje nuevo del usuario. Devolvemos
    # un intent neutro para que el enrutador termine el flujo sin volver
    # a planear o modificar.
    if isinstance(state["messages"][-1], ToolMessage):
        return {"intent": "chat"}

    # ... resto del nodo igual ...
```

Finalmente, reemplaza **respond_node** por esta versión que bindea los frontend tools y también corta si el último mensaje ya es un tool result:

Python

```
def respond_node(state: ItineraryState) -> dict:
    # Si el último mensaje es un ToolMessage, el agente ya invocó la tool
    # del frontend en este turno. No generes otra respuesta: cerraría el
    # ciclo con otra tool call y el grafo entraría en bucle.
    if state["messages"] and isinstance(state["messages"][-1], ToolMessage):
        return {}

    intent = state.get("intent")
    if intent == "new_plan":
        task = RESPOND_TASK_NEW_PLAN.format(
            num_days=state["num_days"], city=state["city"])
    elif intent == "modify":
        task = RESPOND_TASK_MODIFY.format(
            modification=state.get("modification"))
    else:
        task = RESPOND_TASK_DEFAULT

    humans = [m for m in state["messages"] if isinstance(m, HumanMessage)]
    if not humans:
        return {"messages": [AIMessage(content=RESPOND_NO_HUMAN_REPLY)]}

    # Leemos las herramientas que el frontend ha registrado y se las
    # bindeamos al modelo para que pueda llamarlas.
    frontend_tools = state.get("copilotkit", {}).get("actions", [])
    llm_with_tools = llm.bind_tools(frontend_tools) if frontend_tools else llm

    reply = llm_with_tools.invoke([
```

```

    SystemMessage(content=RESPOND_SYSTEM_PROMPT.format(
        city=state.get("city") or "not set",
        num_days=state.get("num_days") or "not set",
        has_itinerary=bool(state.get("itinerary")),
        task=task,
    )),
    HumanMessage(content=humans[-1].content),
])
# Enviamos toda la salida
return {"messages": [reply]}

```

Reinicia ambos servidores y prueba:

- **"Planea un viaje de 3 días a Santa Marta"** → el agente debe responder con un mensaje y debajo aparece la **TripSummaryCard** con Santa Marta, 3, y la cantidad correcta de actividades.
- **"Quita la primera actividad del día 2"** → el itinerario se actualiza en el panel y el chat muestra una nueva **TripSummaryCard** con el total de actividades disminuido.

Checkpoint: La tarjeta aparece automáticamente sin que tú la pidas explícitamente. Los números (**numDays**, **totalActivities**) cuadran con el panel izquierdo. Si pides un chat genérico ("**¿cuál es la mejor época para Roma?**"), la tarjeta no aparece.